

# Simple-Sampling and Hard-Mixup with Prototypes to Rebalance Contrastive Learning for Text Classification

Mengyu Li<sup>\*†</sup>  
Yonghao Liu<sup>\*†</sup>  
College of Computer Science and  
Technology, Jilin University  
Changchun, China  
mengyul21@mails.jlu.edu.cn  
yonghao20@mails.jlu.edu.cn

Fausto Giunchiglia  
Department of Information  
Engineering and Computer Science,  
University of Trento  
Trento, Italy  
fausto.giunchiglia@unitn.it

Ximing Li<sup>\*</sup>  
College of Computer Science and  
Technology, Jilin University  
Changchun, China  
liximing86@gmail.com

Xiaoyue Feng<sup>\*‡</sup>  
College of Computer Science and  
Technology, Jilin University  
Changchun, China  
fengxy@jlu.edu.cn

Renchu Guan<sup>\*‡</sup>  
College of Computer Science and  
Technology, Jilin University  
Changchun, China  
guanrenchu@jlu.edu.cn

## Abstract

Text classification is a crucial and fundamental task in web content mining. Compared with the previous learning paradigm of pre-training and fine-tuning by cross entropy loss, the recently proposed supervised contrastive learning approach has received tremendous attention due to its powerful feature learning capability and robustness. Although several studies have incorporated this technique for text classification, some limitations remain. First, many text datasets are imbalanced, and the learning mechanism of supervised contrastive learning is sensitive to data imbalance, which may harm the model's performance. Moreover, these models leverage separate classification branches with cross entropy and supervised contrastive learning branches without explicit mutual guidance. To this end, we propose a novel model named SharpReCL for imbalanced text classification tasks. First, we obtain the prototype vector of each class in the balanced classification branch to act as a representation of each class. Then, by further explicitly leveraging the prototype vectors, we construct a proper and sufficient target sample set with the same size for each class to perform the supervised contrastive learning procedure. The empirical results show the effectiveness of our model, which even outperforms popular large language models across several datasets. Our code is available [here](#).

## CCS Concepts

- **Computing methodologies** → **Natural language processing**;
- **Information systems** → **Document representation**.

<sup>\*</sup>Key Laboratory of Symbolic Computation and Knowledge Engineering of the Ministry of Education

<sup>†</sup>Equal Contribution

<sup>\*</sup>Corresponding Author



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792237>

## Keywords

Text Classification, Contrastive Learning, Imbalanced Learning

### ACM Reference Format:

Mengyu Li, Yonghao Liu, Fausto Giunchiglia, Ximing Li, Xiaoyue Feng, and Renchu Guan. 2026. Simple-Sampling and Hard-Mixup with Prototypes to Rebalance Contrastive Learning for Text Classification. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792237>

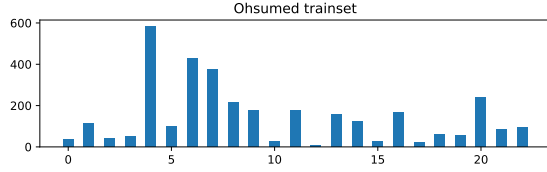
## 1 Introduction

Text classification (TC), as one of the fundamental tasks in the field of web content mining, has attracted much attention from academia and industry and is widely used in various fields [31], such as question answering [28] and topic labeling tasks [3]. In TC processes, the ability of the model to extract textual features from raw text is crucial to the model performance [29, 30, 32]. Text data in the form of symbolic sequences are extremely different from image data, and their discrete nature makes feature extraction and information utilization time-consuming and inefficient.

Owing to the remarkable performance of large-scale pre-trained models, the learning paradigm of pre-training and fine-tuning has emerged as the new standard across various fields. In web content classification tasks, numerous models adhere to this paradigm and are fine-tuned under the guidance of cross-entropy (CE) loss to adapt to downstream tasks [10, 33]. Despite achieving state-of-the-art results on many datasets, CE loss exhibits certain limitations: a number of studies [5, 33] have demonstrated that CE results in reduced generalization capabilities and lacks robustness to data noise [40]. Furthermore, CE exhibits instability across different runs when employed for fine-tuning in classification tasks, potentially impeding the model from attaining optimal performance [12].

Recently, the emerging contrastive learning (CL) has showcased impressive representation learning capabilities and has been extensively applied in both computer vision (CV) and natural language processing (NLP) domains [8, 14, 16]. Notably, supervised contrastive learning (SCL) [20], a supervised variant of CL, aims

to incorporate label information to extend the unsupervised In-foNCE loss to a supervised contrastive loss. This approach brings the representations of samples belonging to the same class closer together while pushing those of different classes apart. Some work [17] has demonstrated that SCL is less sensitive to hyperparameters in optimizers and data augmentation, exhibiting enhanced stability. Consequently, several studies [7, 15] have attempted to merge the learning methods of SCL and cross-entropy loss for fine-tuning PLMs, achieving considerable success.



**Figure 1: Label distribution of the Ohsumed dataset. The horizontal coordinate denotes the category, and the vertical coordinate denotes the corresponding frequency.**

Despite the fruitful success of previous SCL-based models in TC tasks, there are several limitations that hinder the performance of these models. First, they all assume that the models are trained on large-scale, sufficient and balanced datasets, where each class has adequate and equal training samples [55]. However, in real-world application scenarios, the label distribution of data is often imbalanced. Especially when there are many classes in the dataset, many of which (minority class) have few samples, and a handful (majority class) have numerous samples. As shown in Fig. 1, existing TC datasets such as the Ohsumed dataset [51] are often class-imbalanced. When deep models are trained with such datasets, they may yield suboptimal results and even suffer from overfitting problems, especially in minority classes [50, 53]. Meanwhile, the manner of SCL in selecting positive and negative pairs also makes it more sensitive to data imbalance. Specifically, we suppose that the batch is  $B$  and the imbalance ratio of the batch is  $ir = \frac{\pi_{\max}}{\pi_{\min}}$ , where  $\pi_{\max}$  and  $\pi_{\min}$  are the frequencies of the majority class and the minority class in the original dataset, respectively. Since each instance should select samples from the same class to form the positive pairs and from other classes to form the negative pairs, the imbalance ratio can be further written as  $ir = \frac{(|B|\pi_{\max})^2}{(|B|\pi_{\min})^2} = \left(\frac{\pi_{\max}}{\pi_{\min}}\right)^2$  (the detailed analysis is shown in the “Preliminaries” section). That means when performing SCL on imbalanced datasets, the imbalance ratio is quadratic, and the imbalanced issue becomes more severe. Moreover, when the class imbalance problem exists in the dataset, samples may not encounter the proper positive or negative samples within a batch in the SCL paradigm [39]. That is, the majority class samples may lack sufficient negative samples and the minority class samples can also lack adequate positive samples. Therefore, obtaining excellent model performance when performing SCL for imbalanced TC tasks is a great challenge, and there has been little effort to explore it.

Second, previous models [15, 41] commonly utilize labeled data in two different branches, *i.e.*, learning separately by following the paradigm of CE and of SCL and then obtaining the final training loss by a weighted average of the corresponding losses. However,

such approaches only superficially combine CE and SCL, without any component or architecture linking CE and SCL to make them carry out further display interactions. It is also worth noting that the labeled data utilized for CE and SCL are the same, and these two modules both aim to achieve the effect of aggregating samples from the same class while separating samples from different classes. Therefore, further extracting effective information from these two supervised learning modules in an interactive manner to guide model training remains a challenge.

To address the above issues, we propose SharpReCL, meaning Simple-Sampling and Hard-Mixup with Prototypes to Rebalance Contrastive Learning for TC. To address the problem of lack of mutual guidance between the two learning branches, we leverage the learned weight of classification branch to compute class prototype vectors, which serves as a bridge connecting to the SCL branch, where they act as anchor data and guide the generation of hard samples. Specifically, we replenish these prototype vectors as samples of the corresponding classes for each training batch, which ensures that every class can be sampled at least once, thus avoiding the situation where samples from the minority classes are not sampled. Moreover, for the class-imbalanced problem, in addition to resampling the original samples, we first identify those hard-to-distinguish sample embeddings of each class from the original data according to the similarity calculated with the prototype vectors. Next, to improve the diversity of contrastive pairs and further supplement the data, we innovatively use mixup techniques [52] to simultaneously generate *hard negative* and *hard positive* samples with hard-to-distinguish embeddings, rather than just relying on hard negative mining in SCL. With the embeddings of one training batch of original data, the synthetic data constitute a balanced target sample set for SCL learning and create difficult contrastive pairs for model training to achieve better performance. Our key contributions are as follows:

- (1) We propose a novel model, namely, SharpReCL, to efficiently handle imbalanced TC tasks and to overcome the existing challenges of SCL-based models.
- (2) Our model employs simple-sampling and hard-mixup techniques to generate a balanced sample set for SCL to mitigate the class imbalance problem. Moreover, the SCL and classification branches interact and guide each other via prototype vectors for model training.
- (3) We conduct extensive experiments on several benchmark datasets and SharpReCL achieves excellent results on several imbalanced datasets even compared to LLMs, showing its superiority in handling imbalanced data.

## 2 Related Work

**Text Classification.** As one of the fundamental tasks in web content mining, TC has been applied to many fields, including sentiment analysis, topic labeling, etc. The development of deep learning has brought about a revolution in TC models, where neural network-based models can automatically extract text features compared to traditional methods based on tedious feature engineering. CNN [21] and RNN [27], as two representative models of neural networks, have achieved good performance in TC tasks. However, the performance of such models is still limited due to the long-term

dependency problem. PLMs based on Transformer architectures [45], such as BERT [10] and RoBERTa [33], that use a large-scale corpus for pretraining have shown superior performance in various downstream tasks. Recently, LLMs, such as GPT-3.5 [36] and Llama [43] trained on massive high-quality datasets unify various NLP tasks into the text generation paradigm and achieve outstanding performance in text understanding.

**Contrastive Learning.** In recent years, self-supervised representation learning has made impressive progress. As a representative approach in this field, CL has been successfully used in several domains [8, 14]. The idea of CL is to align the features between positive pairs while the features of negative pairs are mutually exclusive to achieve effective representation learning. SCL [20] generalizes unsupervised contrastive approaches to fully-supervised settings to leverage the available label information. Recent works have successfully introduced SCL into TC tasks. For example, DualCL [7] adopts a dual-level CL mechanism to construct label-aware text representations by inserting labels directly in top of the original texts. However, SCL performs poorly when directly applied to imbalanced datasets.

**Imbalanced Learning.** Previous methods specific to the class-imbalanced problem are mainly divided into two categories: reweighting [4, 54] and resampling [2, 19]. The reweighting methods, such as focal loss [25] and dice loss [35], assign different weights to different classes, which means that samples from the minority classes are assigned higher weights while those from the majority classes receive relatively lower weights. The resampling approach, such as SMOTE [6], is mainly implemented by undersampling the majority classes and oversampling the minority classes. Moreover, logit compensation [34] uses class occurrence frequency as prior knowledge, thus achieving effective classification.

### 3 Methods

In this section, we first define of the problem and the basics of CL. Next, we analyze why CL further aggravates the imbalance problem. Finally, we introduce details of the proposed SharpReCL.

#### 3.1 Preliminaries

**Problem Definition:** For multiclass TC tasks with  $C$  classes, we assume the given dataset  $\{x_i, y_i\}_{i=1}^N$  contains  $N$  training examples, where  $x_i$  is the text sequence and  $1 \leq y_i \leq C$  is the label assigned to the input text. Our target is to obtain a function  $\phi$  mapping from an input space  $\mathcal{X}$  to the target space  $\mathcal{Y}$ . Usually, the function  $\phi$  is implemented as the composition of an encoder  $\Psi: \mathcal{X} \rightarrow \mathcal{Z} \in \mathbb{R}^h$  and a linear classifier  $\Omega: \mathcal{Z} \rightarrow \mathcal{Y}$ . Our model also adopts this structure. **Contrastive Learning:** Given  $N$  training samples  $\{x_i\}_{i=1}^N$  and their corresponding augmented samples, each sample has at least one augmented sample in the dataset. For an instance  $x_i$  and its representation  $z_i$  in a batch  $B$ , the standard unsupervised CL loss [8] can be presented as:

$$\mathcal{L}_i^{\text{UCL}} = -\frac{1}{|B|} \log \frac{\exp(z_i \cdot z_i^+ / \tau)}{\sum_{k \in B \setminus \{i\}} \exp(z_i \cdot z_k / \tau)}, \quad (1)$$

where  $z_i^+$  denotes the representation of the augmented sample derived from the  $i$ -th sample  $x_i$ , and  $\tau > 0$  is the temperature factor that controls tolerance to similar samples.

SCL treats all samples with the same label in the batch as positive and all other samples as negative. Its loss function can be written as:

$$\mathcal{L}_i^{\text{SCL}} = \frac{1}{|B_y| - 1} \sum_{p \in B_y \setminus \{i\}} -\log \frac{\exp(z_i \cdot z_p / \tau)}{\sum_{k \in B \setminus \{i\}} \exp(z_i \cdot z_k / \tau)}, \quad (2)$$

where  $B_y$  denotes a subset of  $B$  that contains all samples of class  $y$ .

**Analysis:** We suppose the distribution of each class in the original dataset of  $N$  samples is  $\{\pi_i\}$ ,  $1 \leq i \leq C$ . Generally, the imbalance rate in the dataset can be defined as:

$$\text{ir} = \frac{\max\{\pi_i\}}{\min\{\pi_i\}} = \frac{\pi_{\max}}{\pi_{\min}}. \quad (3)$$

Since SCL works at the sample-pair level, in each batch  $B$ , we define  $P_i$  as the number of sample pairs that belong to the same class  $i$  and participate in CL. Accordingly, we can define the contrastive imbalance rate as follows:

$$\begin{aligned} \text{ir}^{\text{SCL}} &= \frac{\max\{P_i\}}{\min\{P_i\}} = \frac{|B|\pi_{\max} \cdot (|B|\pi_{\max} - 1)}{|B|\pi_{\min} \cdot (|B|\pi_{\min} - 1)} \\ &\approx \frac{(|B|\pi_{\max})^2}{(|B|\pi_{\min})^2} = \left(\frac{\pi_{\max}}{\pi_{\min}}\right)^2 = \text{ir}^2. \end{aligned} \quad (4)$$

Thus, we find that when SCL is performed, for every batch, the imbalance ratio becomes the square of the original ratio. Since each epoch consists of many batches, assuming that the sampling in each batch is uniform, the imbalance ratio of each epoch also remains  $\text{ir}^2$ . In summary, since SCL works with sample pairs, such a learning mechanism will significantly amplify the class-imbalanced problem in the original data and create more difficulties for model training.

#### 3.2 Overview of SharpReCL

The framework of our model is shown in Fig. 2. Our model consists of two branches, namely, the balanced classification branch and the balanced CL branch. We first apply conventional data augmentation techniques in NLP (such as back-translation [13], random noise injection [48], and word substitution [47]) to obtain the augmented data  $D_{\text{aug}}$  for the original data  $D_{\text{org}}$ . We default to word substitution, and the results of different augmentation methods can be found in **Appendix A.1**. Here, we denote the texts under two perspectives as  $D = D_{\text{org}} \cup D_{\text{aug}}$ . Since PLMs such as BERT have shown their superior representation capability, we use PLMs as the backbone encoder for text samples in SharpReCL. For each text sequence sample  $x_i$ , we first obtain its encoded representations using the encoder, i.e.,  $\text{feat}_i = \text{PLM}(x_i) \in \mathbb{R}^{h_1}$ . Then, the balanced classification branch performs TC with the derived text representations and obtains class prototype vectors. After that, we project these two types of representations into the same space. In this space, the data are expanded by mixup techniques, and then, the balanced CL is performed.

#### 3.3 Balanced Classification Branch for Class Prototype Generation

The prototype vector represents each class and acts as a data complement by becoming an instance of the corresponding class in subsequent operations. In SharpReCL, we use the class-specific weights from the backbone classification branch to represent each

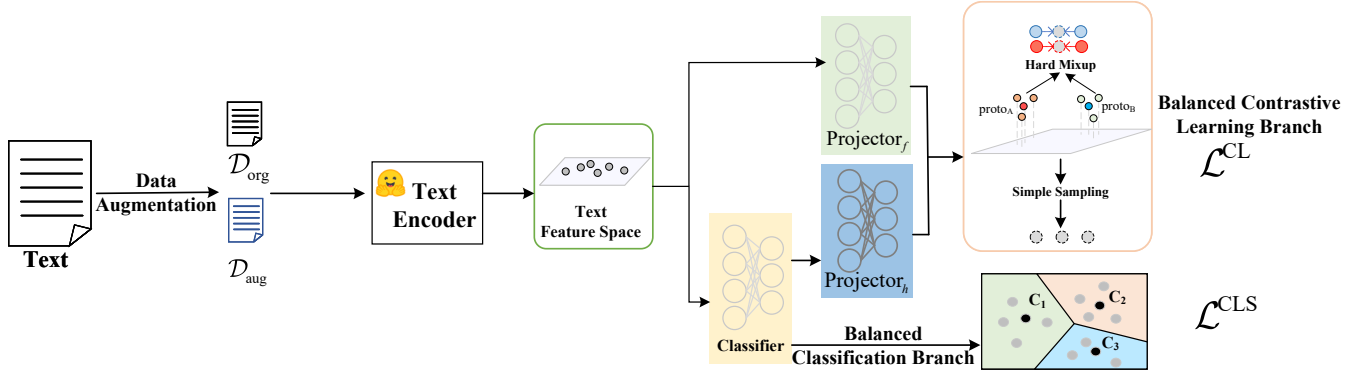


Figure 2: Architecture of SharpReCL (Best viewed in color).

class, making the class prototype vectors directly learnable. Moreover, these prototype vectors can be corrected by explicitly utilizing labels in the classification task.

Most classification models typically adopt cross-entropy loss, which is suboptimal on imbalanced data since the label distributions introduce bias into the trained model. These models perform well for instances of the majority class but poorly for instances of the minority class. Logit compensation [34] is an effective method for imbalanced data that overcomes the shortcomings of the previous reweighting methods and increases the classification boundary between classes based on a simple adjustment of the traditional cross-entropy loss with a class prior. Therefore, we adopt logit compensation instead of the conventional cross-entropy loss in our experiments.

Here, we use a linear function to obtain the class logits, where the weights of the linear function are used as class-specific weights. After the procedure of one projection head  $\text{proj}_h$ , we can obtain the prototype vectors. The above operations can be expressed as:

$$\varphi(\text{feat}) = \mathbf{w}^T \cdot \text{feat}, \quad \text{proto}_{1:C} = \text{proj}_h(\mathbf{w}), \quad (5)$$

where  $\varphi(\text{feat}) \in \mathbb{R}^{N \times C}$  denotes the class logits, and  $\text{proto}_{1:C} \in \mathbb{R}^{C \times h_2}$  represents the class prototype vectors. The projection head  $\text{proj}_h$  is implemented by a two-layer MLP. After obtaining class logits, we perform logit compensation for classification, which can be written as:

$$\mathcal{L}^{\text{CLS}}(y, \varphi(\text{feat})) = -\log \frac{\exp(\varphi_y(\text{feat}) + \delta_y)}{\sum_{y' \in [Y]} \exp(\varphi_{y'}(\text{feat}) + \delta_{y'})}. \quad (6)$$

Here,  $\delta_y$  stands for the compensation for class  $y$ , and its value is related to the class-frequency. In SharpReCL, we set the class compensation  $\delta_y$  to  $\log P_y$ , where  $P_y$  is the class prior of class  $y$ .

### 3.4 Simple-Sampling and Hard-Mixup for the Rebalanced Dataset

We use the feature projection head  $\text{proj}_f$  to map the encoded representations  $\text{feat}$  of texts such that the texts' hidden representation  $\mathbf{z}$  is in the same space as the prototype vector  $\text{proto}$ . For each text-encoded representation  $\text{feat}_i$ , this operation can be denoted as:

$$\mathbf{z}_i = \text{proj}_f(\text{feat}_i). \quad (7)$$

Since prototype vectors are used to represent each class, we address the extreme case where samples from the minority classes do not appear in a batch. This is because we supplement the prototype vectors into each batch of data. We denote the extended dataset as  $\hat{D} = Z \cup \text{proto}$  and then perform  $l_2$ -normalization on it, where  $Z$  and  $\text{proto}$  represent the sets of hidden text vectors and class prototype vectors, respectively. In this section, we further correct the imbalanced dataset by constructing a balanced dataset.

On the one hand, not all samples are helpful for model training, and overly easy samples contribute little to the gradient [37]; on the other hand, extreme (particularly hard) samples can also lead to performance degradation [39]. Therefore, in SharpReCL, we use a combination of simple-sampling and hard-mixup to construct the balanced dataset.

**Simple-Sampling:** As a natural idea, we need to perform resampling on the dataset to expand it. In SCL, positive samples of each class are samples from the same class, and negative samples are other samples from different classes. Therefore, for each class  $c$ ,  $1 \leq c \leq C$ , we can construct its positive sample set  $D^+$  and negative sample set  $D^-$  by simply sampling from  $\hat{D}$ , which can be written as:

$$\begin{aligned} D_{\text{samp},c}^+ &= \text{Samp}\{\mathbf{z}_i\}, \mathbf{z}_i \in \hat{D} \wedge y_i = c, \\ D_{\text{samp},c}^- &= \text{Samp}\{\mathbf{z}_i\}, \mathbf{z}_i \in \hat{D} \wedge y_i \neq c, \end{aligned} \quad (8)$$

where  $\text{Samp}\{\cdot\}$  denotes the sampling operation.

**Hard-Mixup:** We use the prototype vectors as a criterion to measure the learning difficulty of the samples to obtain the hard sample set. For each class  $c$ ,  $1 \leq c \leq C$  and its prototype vector  $\text{proto}_c$ , we consider the top- $k$  positive samples from class  $c$  that are not similar to  $\text{proto}_c$  as hard positive samples, and the top- $k$  negative samples from other classes that are similar to  $\text{proto}_c$  as hard negative samples, which can be denoted as:

$$\begin{aligned} D_{\text{hard},c}^+ &= \text{top-}k\{-\text{proto}_c \cdot \mathbf{z}_i, \mathbf{z}_i\}, \mathbf{z}_i \in \hat{D} \wedge y_i = c, \\ D_{\text{hard},c}^- &= \text{top-}k\{\text{proto}_c \cdot \mathbf{z}_i, \mathbf{z}_i\}, \mathbf{z}_i \in \hat{D} \wedge y_i \neq c. \end{aligned} \quad (9)$$

To increase sample diversity and to improve learning efficiency, we need to further amplify the hard samples obtained by previous procedures. In SharpReCL, we apply linear interpolation to further

augment the data in hard sample sets. This operation is denoted as:

$$\begin{aligned} D_{\text{syn},c}^+ &= \left\{ \frac{\tilde{z}_k^+}{\|\tilde{z}_k^+\|} \right\}, \tilde{z}_k^+ = \alpha_k z_i^+ + (1 - \alpha_k) z_j^+, \\ D_{\text{syn},c}^- &= \left\{ \frac{\tilde{z}_k^-}{\|\tilde{z}_k^-\|} \right\}, \tilde{z}_k^- = \beta_k z_i^- + (1 - \beta_k) z_j^-, \end{aligned} \quad (10)$$

where  $(z_i^+, z_j^+)$  and  $(z_i^-, z_j^-)$  are randomly sampled from the corresponding hard sample sets  $D_{\text{hard},c}^+$  and  $D_{\text{hard},c}^-$ , and  $\alpha_k, \beta_k \sim \text{Beta}(\lambda, \lambda)$  are in the range of  $(0, 1)$ . By default, we set  $\lambda$  to 0.5.

In this way, for each class  $c$ , we construct a balanced positive sample set  $\tilde{D}_c^+ = D_{\text{samp},c}^+ \cup D_{\text{syn},c}^+$  and a balanced negative sample set  $\tilde{D}_c^- = D_{\text{samp},c}^- \cup D_{\text{syn},c}^-$ . The rebalanced dataset  $\tilde{D}$  for all classes can be represented as  $\tilde{D} = \{\tilde{D}_c^+ \cup \tilde{D}_c^-\}_c = D_{\text{samp}} \cup D_{\text{syn}}$ .

### 3.5 Contrastive Learning Branch with a Rebalanced Dataset

One issue of concern is the control of the ratio of the hard-mixup dataset  $D_{\text{syn}} = \{D_{\text{syn},c}^+ \cup D_{\text{syn},c}^-\}_c$  to the balanced dataset  $\tilde{D}$ . Since the feature extraction ability of the model becomes stronger with increasing iterations during the training process, we set the ratio for  $D_{\text{syn}}$  to  $0.5 + \frac{t}{T} \times 0.5$ , with  $t$  being the current number of iterations and  $T$  being the total number of iterations. In this way, the difficulty of the balanced sample set we construct is flexible and controllable, which becomes more challenging during the training process and facilitates the training and learning of the model.

Note that we treat the sample pairs in traditional CL as two parts, *i.e.*, the anchor sample and the target sample. The anchor sample comes from the original samples in the batch together with prototype vectors, *i.e.*,  $\hat{D}$ , and the target sample set consists of  $\hat{D}$  and our constructed sample set  $\tilde{D}$ . Therefore, the original class imbalance problem still exists in the anchor samples.

To alleviate this problem, we add the class prior  $\delta$  used in Eq. 6 to the original SCL to correct  $\mathcal{L}^{\text{SCL}}$  in Eq. 2 based on the idea of reweighting. The concrete objective function can be written as:

$$\begin{aligned} \mathcal{L}_i^{\text{CL}} &= -\frac{\delta_y}{|\hat{D}|} \sum_{p \in \tilde{D}_y^+ \cup \hat{D}_y \setminus \{i\}} \log \frac{\exp(z_i \cdot z_p / \tau)}{\sum_{k \in \tilde{D}_y \cup \hat{D} \setminus \{i\}} \exp(z_i \cdot z_k / \tau)}, \\ \mathcal{L}^{\text{CL}} &= \sum_{i \in \hat{D}} \mathcal{L}_i^{\text{CL}}. \end{aligned} \quad (11)$$

The generation of hard samples can alleviate the issue of vanishing gradients and is beneficial for CL training, as supported by mathematical analysis in **Section 3.6**.

Finally, we have the following loss for training:

$$\mathcal{L}^{\text{overall}} = \mathcal{L}^{\text{CLS}} + \mu \mathcal{L}^{\text{CL}}, \quad (12)$$

where  $\mu$  is the hyperparameter that controls the impact of CL branch, and this branch only intends for the backbone to learn the desired feature embeddings. We feed the obtained test text embeddings to the classification branch during the model inference stage to evaluate the model performance. Moreover, both the classification branch and the CL branch influence the learnable prototype vectors, connecting to each other through these vectors and facilitating explicit interaction and mutual guidance.

The concrete training procedure of our model can be found in Algorithm 1.

---

#### Algorithm 1 SharpReCL Training

---

**Input:** The text dataset  $D = D_{\text{org}} \cup D_{\text{aug}} = \{x_i, y_i\}_i^{2N}$ , the text encoder  $PLM(\cdot)$ , the linear function  $\varphi(\cdot)$ , projection heads  $proj_f(\cdot)$  and  $proj_h(\cdot)$

- 1: Initialize the network parameters
- 2: **for**  $t = 1, 2, \dots, T$  **do**
- 3:   Obtain texts' encoded embeddings  $feat$  using  $PLM(\cdot)$
- 4:   Obtain the class logits and prototype vectors  $proto$  with  $\varphi(\cdot)$  and  $proj_h(\cdot)$  in Eq.5
- 5:   Obtain hidden embeddings of texts with  $proj_f(\cdot)$  in Eq.7
- 6:   Compute  $|D_{\text{samp}}|$  and  $|D_{\text{syn}}|$
- 7:   Perform Simple-Sampling to obtain  $D_{\text{samp}}$  with Eq.8
- 8:   Perform Hard-Mixup to obtain  $D_{\text{hard}}$  with Eq.9
- 9:   Apply Mixup techniques to obtain  $D_{\text{syn}}$  with Eq.10
- 10:   Obtain the re-balanced dataset  $\tilde{D} = D_{\text{samp}} \cup D_{\text{syn}}$
- 11:   Compute the classification loss  $\mathcal{L}^{\text{CLS}}$  with Eq.6
- 12:   Compute the CL loss  $\mathcal{L}^{\text{CL}}$
- 13:   Compute the overall loss  $\mathcal{L}^{\text{overall}}$  with Eq.12
- 14:   Optimize the model by backpropagation of  $\mathcal{L}^{\text{overall}}$
- 15: **end for**

**Output:** The well-trained SharpReCL

---

### 3.6 Mathematical Analysis with Hard Samples

In this section, we provide a detailed mathematical analysis of why generating hard samples can alleviate the vanishing gradient problem and benefit CL training.

For the used supervised contrastive loss as shown in Eq. 11, we can obtain the gradient of  $\mathcal{L}_i^{\text{CL}}$  with respect to node embeddings  $z_k$  as follows:

$$\begin{aligned} \frac{\partial \mathcal{L}_i^{\text{CL}}}{\partial z_i} &= \frac{1}{\tau} \left[ \sum_{p \in \tilde{D}_y^+ \cup \hat{D}_y \setminus \{i\}} z_p (P_{i,p} - \frac{1}{|\hat{D}|}) + \sum_{k \in \tilde{D}_y \cup \hat{D} \setminus \{i \cup \hat{D}_y\}} z_k P_{i,k} \right] \\ P_{i,k} &= \frac{\exp(z_i \cdot z_k / \tau)}{\sum_{k \in \tilde{D}_y \cup \hat{D} \setminus \{i\}} \exp(z_i \cdot z_k / \tau)} \end{aligned} \quad (13)$$

According to Eq. 13, the gradient of  $\mathcal{L}_k$  with respect to  $z_k$  consists of two parts. The first part is the gradient provided by positive sample pairs, and the second part is the gradient provided by negative sample pairs. When positive pairs are simple, it causes  $P_{i,p} \rightarrow \frac{1}{|\hat{D}|}$ , whereas when negative pairs are particularly simple, it causes  $P_{i,k} \rightarrow 0$ . Both scenarios can lead to gradient vanishing, causing unstable model training and damaging the learning process of the entire model.

However, we only generate hard positive and negative samples, thus providing larger gradients for the contrastive objective. It helps prevent the aforementioned cases and promotes more stable model training.

## 4 Experiment

**Dataset.** We employ six widely used real-world datasets to evaluate the effectiveness of our model, where the train/test split of datasets is the same as previous studies [42]. They are **R52** [30], **Ohsumed** [26], **TREC** [23], **DBLP** [42], **Biomedical** [49], and **CR**

[11], with detailed descriptions provided in **Appendix A.2**. The detailed statistics of these datasets are summarized in Table 1. We use the original R52 and Ohsumed datasets directly, since they are extremely **imbalanced**. Other datasets are relatively **balanced**. There is only one category in the TREC with few samples. Thus the calculated *ir* is large, but we still treat it as a balanced dataset, which needs preprocessing. We create the imbalanced version of balanced datasets by reducing the training samples of each sorted class with a deterministic *ir*, which is a common practice in imbalanced learning [9]. For example, if we set *ir* to 10, the number of each category in a balanced dataset with three categories is 1000, 900, and 890, each category in the imbalanced version is 1000, 100, and 10.

**Table 1: Detailed statistics of the datasets.**

| Dataset    | #Train | #Test  | #Word  | Avg. Len | #Class | <i>ir</i> |
|------------|--------|--------|--------|----------|--------|-----------|
| R52        | 6,532  | 2,568  | 8,892  | 69.8     | 52     | 1307.7    |
| Ohsumed    | 3,353  | 4,043  | 14,157 | 6.8      | 23     | 65.0      |
| TREC       | 5,452  | 500    | 8,751  | 11.3     | 6      | 14.5      |
| DBLP       | 61,422 | 20,000 | 22,27  | 8.5      | 6      | 4.0       |
| Biomedical | 17,976 | 1,998  | 18,888 | 12.9     | 20     | 1.1       |
| CR         | 3,394  | 376    | 5,542  | 18.2     | 2      | 1.8       |

**Baseline.** We compare the proposed model with the following four types of competitive baselines. *Pretrained language models and their imbalanced learning versions* include **BERT** [10], **BERT+Dice Loss** [24], **BERT+Focal Loss** [25], and **RoBERTa** [33]. *Contrastive learning models* contain **SimCSE** [14], **SCLCE** [15], **SPCL** [39], and **DualCL** [7]. *Imbalanced contrastive learning models* include **Hybrid-SC** [46], **BCL** [55], and **TSC** [22]. *Large language models* consist of **GPT-3.5** [36], **Bloom-7.1B** [38], **Llama2-7B** [43], and **Llama3-8B** [1].

**Implementation Details.** In the SharpReCL, we utilize the pre-trained BERT base-uncased model as the text encoder by using AdamW with a learning rate of  $5e-5$ . The weight decay and batch size are set to  $5e-4$  and 128, respectively. We implement the projection head by an MLP with one hidden layer activated by ReLU. We default to word substitution for augmenting sentences. The temperature  $\tau$  of SCL is chosen from {0.3, 0.5, 1}. Moreover, the number of hard positive and hard negative samples per class in Eq.9 is uniformly 20, i.e.,  $|D_{\text{hard},c}^+| = |D_{\text{hard},c}^-| = 20$ . The number of generated rebalanced positive and negative samples when using mixup for each class  $c$  are 10 and 500, i.e.,  $\tilde{D}_c^+ = 10$  and  $\tilde{D}_c^- = 500$ . The parameter  $\mu$  for controlling the loss function is 1. To accelerate model training, we use two 24GB Nvidia GeForce RTX 3090 GPUs.

For SimCSE, similar to SCLCE, we use a linear combination of unsupervised CL loss and CE loss to fully utilize the labeled data for model training. For RoBERTa, we adopt the RoBERTa-base version. For imbalanced CL methods, they are originally designed for images. To adapt these methods to the texts, we replace their feature extractors with a text encoder (BERT). For LLMs, we fine-tune GPT-3.5 using the training data from the evaluation dataset through the fine-tuning interface provided by OpenAI. For Bloom-7.1B, Llama2-7B, and Llama3-8B, we perform full-scale fine-tuning using the training data. To reduce GPU memory usage, we employ

LoRA [18] and 4-bit quantization techniques through the Parameter-Efficient Fine-Tuning (PEFT) method provided by Hugging Face. The prompts used for text classification in Bloom-7.1B, Llama2-7B, and Llama3-8B are the same as those used for GPT-3.5. For other remaining baselines, we use the open-source codes and adopt the parameters suggested by their original papers.

The detailed descriptions of baselines are provided in **Appendix A.3**. We employ the accuracy (Acc) and macro-f1 score (F1) to examine all the models' performance. All experiments are conducted ten times to obtain average metrics for statistical significance.

## 5 Results

**Model Performance.** We perform extensive experiments on our model and other competitive baselines with six public datasets under different imbalanced settings (R52 and Ohsumed), including original data, *ir*=50, *ir*=20 and *ir*=10, as shown in Table 2. Furthermore, we present the results of our model and other LLMs under *ir*=50 in Table 3. Based on the quantitative results, we gain in-depth insights and analyses. First, our model significantly outperforms other baselines in most imbalanced experimental settings if excluding LLMs, illustrating its superiority in modeling imbalanced text data. An important reason is that SharpReCL introduces class prototypes during training, which ensures that all classes appear in every batch, effectively resolving the problem of missing minority classes due to data sampling. Meanwhile, we introduce hard positive and hard negative samples for each class in SCL, which can provide more adequate gradients and thus facilitate model optimization. Based on this, we generate a balanced contrastive queue for each class by using the mixup technique, which effectively alleviates the previous issue that the majority categories dominate the training process in an imbalanced dataset. Additionally, we enable the two learning branches to explicitly interact and mutually guide each other through the prototype vectors, thereby enhancing model training.

We find other models achieve inferior performance due to serious bias in representation learning. Notably, our model achieves the best performance on several datasets (such as Ohsumed and Biomedical) even when compared to LLMs. One plausible reason is that LLMs lack sufficient domain-specific training data, leading to their inadequate understanding of these texts. We find that SharpReCL does not perform the best on the DBLP compared to LLMs. This can be explained by the fact that, as shown in Table 1, DBLP is significantly larger than other datasets. Consequently, each class in DBLP has enough samples for LLMs to extract meaningful features for downstream classification. Although imbalanced contrastive learning models, such as Hybrid-SC and TSC, demonstrate superior performance compared to other categories of models, they still fall behind our model. This performance gap is primarily due to their failure to explicitly exploit hard positive and hard negative samples. Moreover, the imbalanced learning models (e.g., BERT+Dice Loss and BERT+ Focal Loss) and contrastive learning models (e.g., SCLCE, SPCL and DualCL) have comparable performance across all the datasets; the former is dedicated to imbalanced learning, while the latter enjoy the benefits of CL.

**Ablation Study.** To evaluate the individual effects of the proposed SharpReCL, we perform a series of ablation experiments on all

**Table 2: The results (%) of Acc and Macro-F1 scores of various experimental settings. Our model performance is a statistically significant improvement over the best baseline, with a p-value smaller than 0.001. “Avg. metrics” is the average model performance across multiple datasets. “org\*” represents the case where the classes are relatively balanced. Underline: runner-up.**

| Dataset      | ir   | Metric | BERT                                   | BERT+ Dice Loss                 | BERT+ Focal Loss                | RoBERTa                         | SimCSE                   | SCLCE                                  | SPCL                                   | DualCL                                 | Hybrid-SC                | BCL                      | TSC                             | Ours                                                         |
|--------------|------|--------|----------------------------------------|---------------------------------|---------------------------------|---------------------------------|--------------------------|----------------------------------------|----------------------------------------|----------------------------------------|--------------------------|--------------------------|---------------------------------|--------------------------------------------------------------|
| R52          | org  | Acc F1 | 95.55±0.36<br>82.88±0.28               | 95.48±0.29<br>81.58±0.32        | 95.66±0.25<br>83.19±0.26        | 95.56±0.22<br>82.33±0.35        | 89.35±0.20<br>79.22±0.15 | 95.51±0.35<br>83.64±0.39               | 94.29±0.36<br>83.19±0.32               | 94.16±0.16<br>73.92±0.12               | 95.49±0.51<br>83.26±0.50 | 95.56±0.36<br>83.59±0.29 | 95.64±0.32<br>82.90±0.55        | <b>96.11±0.36</b><br><b>84.83±0.29</b>                       |
| Ohsumed      | org  | Acc F1 | 65.71±0.30<br>57.36±0.31               | 65.94±0.20<br>55.59±0.36        | 65.22±0.26<br>55.00±0.28        | 66.06±0.23<br>56.95±0.26        | 49.30±0.32<br>36.00±0.40 | 66.59±0.35<br>56.25±0.39               | 67.20±0.42<br>56.42±0.42               | 26.44±0.35<br>14.75±0.55               | 66.32±0.42<br>58.79±0.66 | 66.39±0.62<br>58.85±0.72 | 66.66±0.72<br>57.92±0.69        | <b>68.44±0.22</b><br><b>60.82±0.20</b>                       |
| TREC         | org* | Acc F1 | 97.60±0.39<br>96.99±0.52               | 97.60±0.42<br>96.97±0.55        | 97.20±0.56<br>96.13±0.65        | <u>97.61±0.55</u><br>97.00±0.62 | 88.80±0.62<br>85.90±0.72 | 97.60±0.38<br>97.01±0.42               | 96.50±0.95<br>95.50±1.20               | <b>97.80±1.25</b><br><u>97.38±1.72</u> | 94.21±1.59<br>96.92±1.86 | 95.60±1.65<br>96.36±1.96 | 96.62±1.62<br>97.09±1.92        | 97.20±0.95<br><b>97.63±0.82</b>                              |
|              | 50   | Acc F1 | 93.40±0.66<br>93.30±0.65               | 93.60±0.50<br>92.36±0.69        | 94.40±0.52<br><u>94.26±0.40</u> | 92.60±0.39<br>92.50±0.41        | 74.00±0.29<br>76.00±0.32 | 94.20±0.30<br>93.97±0.39               | 93.52±1.25<br>92.40±1.52               | 92.80±1.55<br>91.05±1.62               | 93.62±1.99<br>93.79±2.01 | 93.40±1.76<br>92.00±2.12 | 94.02±2.25<br>93.46±2.29        | <b>94.80±0.55</b><br><b>94.83±0.22</b>                       |
|              | 20   | Acc F1 | 96.60±0.49<br>96.05±0.56               | 96.60±0.28<br>96.25±0.19        | 95.80±0.46<br>95.53±0.55        | 96.60±0.53<br>95.31±0.52        | 82.20±0.35<br>82.00±0.38 | 96.80±0.60<br><u>96.32±0.71</u>        | 95.52±0.62<br>94.19±0.63               | 96.40±1.79<br>95.25±1.59               | 95.79±2.15<br>95.86±1.90 | 95.90±2.39<br>95.25±2.25 | 96.60±2.06<br>96.17±2.42        | <b>97.00±0.52</b><br><b>96.45±0.21</b>                       |
|              | 10   | Acc F1 | 96.60±0.42<br>95.38±0.55               | 97.00±0.30<br>95.99±0.37        | 96.80±0.27<br>96.41±0.29        | 97.00±0.24<br>96.55±0.46        | 85.00±0.25<br>82.20±0.70 | 97.00±0.72<br>95.64±0.82               | 96.70±0.75<br>95.72±0.71               | <b>97.60±1.66</b><br><u>97.34±1.72</u> | 96.12±1.22<br>96.09±1.51 | 95.40±2.12<br>95.89±2.22 | 96.80±1.62<br>96.32±1.55        | 97.20±0.35<br><b>97.53±0.39</b>                              |
| DBLP         | org* | Acc F1 | 79.62±0.66<br>76.47±0.18               | 80.09±0.38<br>77.27±0.15        | 79.73±0.53<br>76.57±0.25        | 79.85±0.32<br>76.99±0.45        | 68.50±0.39<br>64.40±0.23 | 79.96±0.59<br>76.84±0.60               | 79.85±0.66<br>77.20±0.68               | <b>80.52±1.73</b><br><u>77.67±1.22</u> | 77.96±1.55<br>77.16±1.29 | 78.76±2.05<br>76.08±2.39 | 78.20±0.85<br>76.19±1.25        | 80.15±0.39<br><u>77.59±0.28</u>                              |
|              | 50   | Acc F1 | 74.53±0.38<br>68.60±0.32               | 74.34±0.39<br>67.20±0.45        | 73.94±0.42<br>67.99±0.39        | 74.19±0.53<br>67.03±0.59        | 62.40±0.30<br>51.30±0.33 | 74.49±0.90<br>67.75±0.92               | 74.96±0.78<br>67.70±0.87               | <u>75.72±1.85</u><br><u>69.57±1.99</u> | 75.19±1.82<br>69.22±2.01 | 75.60±2.19<br>69.10±2.32 | 75.50±2.22<br>69.30±2.55        | <b>76.30±0.25</b><br><b>71.15±0.36</b>                       |
|              | 20   | Acc F1 | 77.20±0.93<br>72.93±1.11               | 76.95±0.65<br>72.69±0.68        | 77.11±0.16<br>72.71±0.23        | 76.70±0.29<br>72.15±0.35        | 65.10±0.35<br>57.00±0.39 | 77.65±0.26<br>73.16±0.30               | 77.72±0.73<br>73.12±0.76               | <u>78.20±2.01</u><br><u>74.07±1.98</u> | 76.12±2.05<br>72.16±2.15 | 76.40±1.88<br>73.10±1.93 | 76.62±2.16<br>73.25±2.55        | <b>78.72±0.31</b><br><b>75.13±0.38</b>                       |
|              | 10   | Acc F1 | 78.86±0.99<br>75.16±1.22               | 78.35±0.82<br>74.78±0.89        | 78.70±0.36<br>75.30±0.32        | 78.21±0.39<br>74.52±0.55        | 67.10±0.38<br>61.00±0.42 | 78.96±0.86<br><u>75.92±0.88</u>        | 78.42±0.77<br>75.42±0.79               | 79.03±2.66<br>75.81±2.72               | 77.52±2.18<br>73.52±2.02 | 77.92±2.08<br>73.42±2.11 | 78.22±2.60<br>73.06±2.69        | <b>79.59±0.41</b><br><b>76.75±0.46</b>                       |
| Biomedical   | org* | Acc F1 | 71.22±0.82<br>71.10±0.86               | 70.76±0.73<br>70.69±0.75        | 70.59±0.30<br>70.32±0.39        | 70.78±0.62<br>70.56±0.65        | 62.80±0.29<br>61.85±0.32 | <u>71.42±0.60</u><br><u>71.32±0.75</u> | 70.92±0.66<br>69.39±0.72               | 70.90±1.89<br>70.81±1.92               | 69.51±2.29<br>69.66±2.19 | 69.90±2.09<br>70.02±2.19 | 69.39±2.51<br>70.50±2.59        | <b>72.52±0.39</b><br><b>72.65±0.26</b>                       |
|              | 50   | Acc F1 | 59.17±0.56<br>57.72±0.55               | 59.32±0.79<br>58.89±0.82        | 55.52±0.19<br>53.76±0.20        | 58.14±0.51<br>57.44±0.55        | 58.22±0.33<br>56.45±0.36 | 59.28±1.01<br>58.45±1.09               | <u>60.16±0.69</u><br><u>59.05±0.89</u> | 59.42±1.90<br>58.72±1.99               | 57.79±2.35<br>57.62±2.16 | 58.10±2.30<br>58.10±2.55 | 58.92±2.22<br>57.85±2.02        | <b>61.56±0.28</b><br><b>60.78±0.12</b>                       |
|              | 20   | Acc F1 | 64.42±0.35<br>64.09±0.31               | <u>64.48±0.63</u><br>64.06±0.56 | 62.67±0.29<br>61.92±0.22        | 62.82±0.66<br>62.24±0.72        | 54.50±0.26<br>54.30±0.30 | 64.37±0.15<br>63.96±0.21               | 64.19±0.39<br>63.49±0.28               | 63.65±1.93<br>63.06±1.99               | 64.26±2.15<br>63.25±2.01 | 64.32±1.86<br>62.16±2.05 | 63.55±2.11<br>62.96±2.19        | <b>65.47±0.22</b><br><b>64.83±0.26</b>                       |
|              | 10   | Acc F1 | <u>66.90±0.39</u><br><u>66.97±0.31</u> | 66.74±0.70<br>66.69±0.69        | 66.39±0.35<br>66.32±0.36        | 66.73±0.82<br>66.52±0.89        | 58.20±0.30<br>58.30±0.39 | 66.81±0.66<br>66.77±0.66               | 66.10±0.58<br>64.92±0.66               | 66.45±0.86<br>66.50±0.90               | 62.26±2.19<br>61.19±1.98 | 65.80±2.09<br>65.02±2.26 | 66.62±1.99<br>66.22±2.05        | <b>68.11±0.60</b><br><b>68.06±0.55</b>                       |
| CR           | org* | Acc F1 | 92.02±0.56<br>91.39±0.59               | 92.26±0.29<br>91.60±0.19        | 91.02±0.39<br>91.30±0.36        | 92.55±0.72<br>92.03±0.77        | 86.05±0.26<br>85.92±0.32 | 91.22±0.16<br>90.62±0.22               | 91.55±0.26<br>91.36±0.28               | <u>92.65±0.77</u><br>92.16±0.79        | 92.16±2.02<br>90.02±2.19 | 92.55±2.11<br>91.01±2.19 | 92.41±2.09<br>91.03±2.16        | <b>93.09±0.30</b><br><b>92.51±0.22</b>                       |
|              | 50   | Acc F1 | 77.39±1.01<br>70.38±1.06               | <u>80.59±0.26</u><br>75.37±0.32 | 77.93±0.55<br>70.67±0.56        | 78.99±0.56<br>73.83±0.59        | 77.90±0.31<br>71.10±0.35 | 79.69±0.18<br>74.46±0.22               | 80.25±0.25<br>75.62±0.30               | 77.13±0.70<br>69.50±0.72               | 78.55±2.05<br>75.29±2.02 | 78.71±2.19<br>75.46±2.25 | 78.92±1.95<br><u>76.41±2.15</u> | <b>84.04±0.25</b><br><b>81.12±0.26</b>                       |
|              | 20   | Acc F1 | 84.04±0.39<br>80.51±0.36               | 83.24±0.55<br>79.48±0.56        | 84.04±0.62<br>80.83±0.65        | 80.59±0.22<br>75.37±0.23        | 81.90±0.31<br>77.40±0.36 | 85.69±0.35<br>83.35±0.36               | 85.42±0.55<br>82.56±0.59               | 88.83±0.60<br><u>87.20±0.66</u>        | 85.55±1.79<br>82.22±1.85 | 84.76±1.99<br>83.60±2.02 | 85.96±1.95<br>82.73±2.15        | <b>89.36±0.50</b><br><b>88.28±0.52</b>                       |
|              | 10   | Acc F1 | 86.44±0.22<br>84.29±0.25               | 88.30±0.26<br>86.60±0.30        | <u>90.69±0.60</u><br>89.53±0.55 | 90.16±0.66<br>88.94±0.69        | 83.80±0.28<br>80.70±0.32 | 88.28±0.31<br>87.29±0.39               | 89.15±0.52<br>88.25±0.55               | 90.43±0.70<br>89.38±0.79               | 89.09±2.10<br>87.56±2.19 | 89.29±1.75<br>87.43±1.89 | 89.02±2.16<br>87.08±2.11        | <b>90.96±0.21</b><br><b>90.21±0.26</b>                       |
| Avg. metrics |      |        | Acc F1                                 | 80.96<br>77.87                  | 81.20<br>78.00                  | 80.75<br>77.68                  | 80.84<br>77.68           | 71.95<br>67.84                         | 81.42<br>78.48                         | 81.25<br>78.08                         | 79.34<br>75.79           | 80.41<br>77.97           | 80.79<br>78.14                  | <u>81.09</u><br><u>78.38</u><br><b>82.81</b><br><b>80.62</b> |

**Table 3: The results of our model and LLMs across different datasets.**

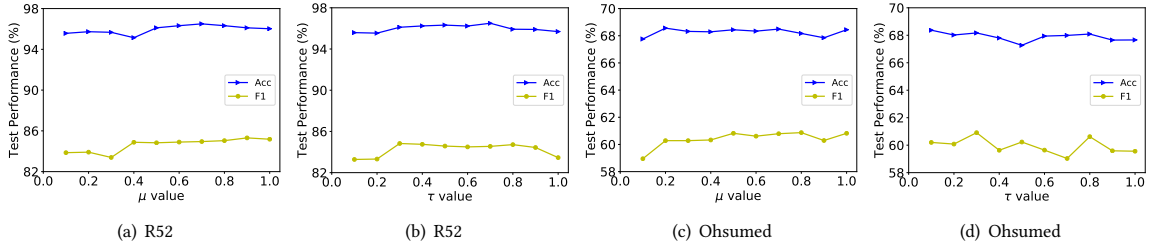
| Dataset    | R52               |                   | Ohsumed           |                   | TREC              |                   | DBLP              |                   | Biomedical        |                   | CR                |                   | Avg. metrics |              |
|------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|--------------|--------------|
| Metric     | Acc               | F1                | Acc               | F1                | Acc               | F1                | Acc               | F1                | Acc               | F1                | Acc               | F1                | Acc          | F1           |
| GPT-3.5    | 95.27±0.55        | 79.15±0.52        | 51.84±0.45        | 43.83±0.65        | 96.62±1.25        | 96.26±1.52        | 78.52±2.39        | 76.52±2.42        | 63.26±2.11        | 63.19±1.98        | 91.09±2.10        | 90.56±2.19        | 80.35        | 77.65        |
| Bloom-7.1B | 95.42±0.36        | 81.29±0.29        | 67.54±0.62        | 53.52±0.72        | <u>97.40±2.12</u> | 96.89±2.22        | 80.92±2.08        | <u>77.42±2.11</u> | 65.80±2.09        | 65.02±2.26        | <u>92.29±1.75</u> | <u>91.43±1.89</u> | 81.29        | 77.32        |
| Llama2-7B  | 95.64±0.32        | 82.90±0.55        | <u>67.66±0.72</u> | 55.92±0.69        | 96.80±1.62        | 96.32±1.55        | 80.22±2.60        | 77.06±2.69        | 66.62±1.99        | 66.22±2.05        | 92.02±2.16        | 91.08±2.11        | 82.00        | 78.56        |
| Llama3-8B  | <u>95.79±0.22</u> | <u>83.72±0.35</u> | 68.02±0.26        | 58.59±0.19        | 95.20±1.66        | 93.59±1.35        | <b>81.26±2.60</b> | <b>77.75±0.46</b> | 62.32±1.90        | 62.82±0.55        | <b>93.35±0.66</b> | <b>92.66±0.59</b> | 81.19        | 78.42        |
| Ours       | <b>96.11±0.36</b> | <b>84.83±0.29</b> | <b>68.44±0.22</b> | <b>60.82±0.20</b> | 97.20±0.35        | <b>97.53±0.39</b> | 79.59±0.41        | 76.75±0.46        | <b>68.11±0.60</b> | <b>68.06±0.55</b> | <b>90.96±0.16</b> | <b>90.21±0.26</b> | <b>82.81</b> | <b>80.62</b> |

datasets except Ohsumed with an imbalance ratio of 50. We use the original Ohsumed dataset. Specifically, *w/o SSHM* removes the simple-sampling and hard-mixup module, which simply incorporates the cross-entropy and CL objective. *w/o SS* and *w/o HM* delete

the simple-sampling module and hard-mixup module, respectively. *w/o CL* excludes the CL branch and leaves the classification branch. *w/o CLS* eliminates the classification branch, which is evaluated with a linear classifier by frozen representations trained via CL. *w/o*

**Table 4: The results of ablation study.**

| Model                     | R52          |              | Ohsumed      |              | TREC         |              | DBLP         |              | Biomedical   |              | CR           |              |
|---------------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
|                           | Acc          | F1           | Acc          | F1           | Acc          | F1           | Acc          | F1           | Acc          | F1           | Acc          | F1           |
| w/o SSHM                  | 94.59        | 82.93        | 66.59        | 56.25        | 94.20        | 93.97        | 74.49        | 67.75        | 59.28        | 58.45        | 79.69        | 74.46        |
| w/o CL                    | 95.09        | 83.52        | 66.86        | 57.52        | 94.40        | 93.30        | 74.65        | 68.04        | 59.44        | 58.98        | 77.39        | 69.75        |
| w/o CLS                   | 95.62        | 83.95        | 66.89        | 57.56        | 94.42        | 93.99        | 74.69        | 68.22        | 59.66        | 60.02        | 78.59        | 72.16        |
| w/o SS                    | 95.66        | 83.99        | 67.28        | 59.13        | 93.80        | 84.88        | 75.53        | 71.43        | 61.35        | 60.72        | 80.05        | 75.15        |
| w/o HM                    | 95.69        | 84.05        | 67.55        | 59.95        | 94.00        | 93.99        | 75.44        | 70.78        | 60.92        | 60.52        | 82.78        | 80.18        |
| w/o MI                    | 95.02        | 83.75        | 67.12        | 59.52        | 93.89        | 93.56        | 74.96        | 69.68        | 60.36        | 59.59        | 83.15        | 79.90        |
| w/o $\delta_{\{CLS,CL\}}$ | 95.32        | 84.33        | 67.86        | 60.22        | 94.40        | 94.43        | 75.39        | 70.05        | 60.82        | 60.32        | 83.49        | 80.51        |
| w/o $\delta_{CLS}$        | 95.79        | 84.59        | 68.19        | 60.46        | 94.46        | 94.55        | 75.52        | 70.26        | 60.93        | 60.46        | 83.52        | 80.68        |
| w/o $\delta_{CL}$         | 95.95        | 84.65        | 68.22        | 60.52        | 94.68        | 94.62        | 75.64        | 70.68        | 61.36        | 60.59        | 83.75        | 80.90        |
| Ours                      | <b>96.11</b> | <b>84.83</b> | <b>68.44</b> | <b>60.82</b> | <b>94.80</b> | <b>94.83</b> | <b>76.30</b> | <b>71.15</b> | <b>61.56</b> | <b>60.78</b> | <b>84.04</b> | <b>81.12</b> |

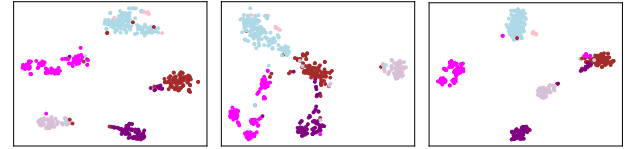
**Figure 3: Sensitivity of SharpReCL with respect to  $\mu$  and  $\tau$  on R52 and Ohsumed datasets.**

MI removes the mutual guidance between the classification branch and the SCL branch, allowing both branches to learn in parallel like other SCL models, simply summing their losses. w/o  $\delta_{\{CLS,CL\}}$  removes the class priors  $\delta$  from both the classification branch shown in Eq.6 and the CL branch shown in Eq.11. w/o  $\delta_{CLS}$  removes the class prior  $\delta$  in the classification branch. w/o  $\delta_{CL}$  eliminates the class prior  $\delta$  in the CL branch. The results are presented in Table 4.

We observe that each component of SharpReCL is indispensable and that removing any of the components degrades the performance. The first variant has the most significant performance degradation because the imbalanced class distribution in a batch makes the model learn biased text representations. The fourth and fifth variants show that simple-sampling and hard-mixup modules are critical for the quality of the rebalanced dataset. Removing the CL branch drastically reduces the ability of the model to extract invariant knowledge from similar texts.

**Parameter Sensitivity.** We investigate the sensitivity of the model with respect to primary hyperparameters on the Ohsumed dataset, i.e., the controlled weight for contrastive loss  $\mu$  and the temperature  $\tau$ . From Fig. 3, we find that the model is insensitive to the parameter  $\mu$  from 0.2 to 1, illustrating its robustness. Moreover, the accuracy of the model is relatively stable across different selections of  $\tau$  from [0.1, 1], while the F1 scores vary drastically depending on different  $\tau$ . We present other hyperparameters in **Appendix A.4**.

**Visualization Study.** To show the effectiveness of our model in learning text representations under an imbalanced scenario, we perform visualizations of the embeddings of test documents by using the t-SNE method [44]. We choose two representative models, SCLCE and SPCL, to compare with our model. From Fig. 4,

**Figure 4: Visualization of different models on TREC under  $ir=50$ . left: SCLCE, middle: SPCL, right: Our method.**

we observe that the embeddings learned by SharpReCL are more discriminative and clustered than others.

## 6 Conclusion

In this work, we propose a novel model, namely, SharpReCL, for imbalanced TC tasks based on SCL. We make the classification branch and the SCL branch communicate and guide each other by leveraging learned prototype vectors. To address the sensitivity of SCL to class imbalance, we construct a balanced dataset consisting of hard negative and hard positive samples by using the mixup technique to balance the sample pairs for SCL training. Extensive experiments on several datasets under different imbalanced settings demonstrate the effectiveness of our model.

## Acknowledgments

Our work is supported by the National Natural Science Foundation of China (No. 62372209). Fausto Giunchiglia’s work is funded by European Union’s Horizon 2020 FET Proactive Project (No.823783).

## References

- [1] AI@Meta. 2024. Llama 3 Model Card. (2024).
- [2] Shin Ando and Chun Yuan Huang. 2017. Deep over-sampling framework for classifying imbalanced data. In *ECMLPKDD*. 770–785.
- [3] Dimosthenis Antypas, Asahi Ushio, Jose Camacho-Collados, Vítor Silva, Leonardo Neves, and Francesco Barbieri. 2022. Twitter Topic Classification. In *COLING*. 3386–3400.
- [4] Jonathon Byrd and Zachary Lipton. 2019. What is the effect of importance weighting in deep learning?. In *ICML*. 872–881.
- [5] Kaidi Cao, Colin Wei, Adrien Gaidon, Nikos Arechiga, and Tengyu Ma. 2019. Learning imbalanced datasets with label-distribution-aware margin loss. *NeurIPS* (2019).
- [6] Nitesh V Chawla, Kevin W Bowyer, Lawrence O Hall, and W Philip Kegelmeyer. 2002. SMOTE: synthetic minority over-sampling technique. *JAIR* 16 (2002), 321–357.
- [7] Qianben Chen, Richong Zhang, Yaowei Zheng, and Yongyi Mao. 2024. Dual Contrastive Learning: Text Classification via Label-Aware Data Augmentation. In *WWW*.
- [8] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 2020. A simple framework for contrastive learning of visual representations. In *ICML*. 1597–1607.
- [9] Yin Cui, Menglin Jia, Tsung-Yi Lin, Yang Song, and Serge Belongie. 2019. Class-balanced loss based on effective number of samples. In *CVPR*. 9268–9277.
- [10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*.
- [11] Xiaowen Ding, Bing Liu, and Philip S Yu. 2008. A holistic lexicon-based approach to opinion mining. In *WSDM*. 231–240.
- [12] Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi, and Noah Smith. 2020. Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping. *arXiv preprint arXiv:2002.06305* (2020).
- [13] Sergey Edunov, Myle Ott, Michael Auli, and David Grangier. 2018. Understanding Back-Translation at Scale. In *EMNLP*. 489–500.
- [14] Tianyu Gao, Xingcheng Yao, and Danqi Chen. 2021. SimCSE: Simple Contrastive Learning of Sentence Embeddings. In *EMNLP*.
- [15] Beliz Gunel, Jingfei Du, Alexis Conneau, and Veselin Stoyanov. 2021. Supervised Contrastive Learning for Pre-trained Language Model Fine-tuning. In *ICLR*.
- [16] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. 2020. Momentum contrast for unsupervised visual representation learning. In *CVPR*. 9729–9738.
- [17] Dan Hendrycks and Thomas Dietterich. 2020. Benchmarking neural network robustness to common corruptions and perturbations. In *ICLR*.
- [18] Edward J Hu, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2021. LoRA: Low-Rank Adaptation of Large Language Models. In *ICLR*.
- [19] Xue Jiang, Feng Liu, Zhen Fang, Hong Chen, Tongliang Liu, Feng Zheng, and Bo Han. 2023. Detecting out-of-distribution data through in-distribution class prior. In *ICML*.
- [20] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschiot, Ce Liu, and Dilip Krishnan. 2020. Supervised contrastive learning. *NeurIPS* (2020), 18661–18673.
- [21] Yoon Kim. 2014. Convolutional Neural Networks for Sentence Classification. In *EMNLP*. 1746–1751.
- [22] Tianhong Li, Peng Cao, Yuan Yuan, Lijie Fan, Yuzhe Yang, Rogerio S Feris, Piotr Indyk, and Dina Katabi. 2022. Targeted supervised contrastive learning for long-tailed recognition. In *CVPR*. 6918–6928.
- [23] Xin Li and Dan Roth. 2002. Learning question classifiers. In *COLING*.
- [24] Xiaoya Li, Xiaofei Sun, Yuxian Meng, Junjun Liang, Fei Wu, and Jiwei Li. 2020. Dice Loss for Data-imbalanced NLP Tasks. In *ACL*. 465–476.
- [25] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2017. Focal loss for dense object detection. In *ICCV*. 2980–2988.
- [26] Hu Linmei, Tianchi Yang, Chuan Shi, Houye Ji, and Xiaoli Li. 2019. Heterogeneous graph attention networks for semi-supervised short text classification. In *EMNLP-IJCNLP*. 4821–4830.
- [27] Pengfei Liu, Xipeng Qiu, Xinchu Chen, Shiyu Wu, and Xuan-Jing Huang. 2015. Multi-timescale long short-term memory neural network for modelling sentences and documents. In *EMNLP*. 2326–2335.
- [28] Yonghao Liu, Mengyu Li Di Liang, Fausto Giunchiglia, Ximing Li, Sirui Wang, Wei Wu, Lan Huang, Xiaoyue Feng, and Renchu Guan. 2023. Local and Global: Temporal Question Answering via Information Fusion. In *IJCAI*.
- [29] Yonghao Liu, Fausto Giunchiglia, Lan Huang, Ximing Li, Xiaoyue Feng, and Renchu Guan. 2025. A Simple Graph Contrastive Learning Framework for Short Text Classification. In *AAAI*.
- [30] Yonghao Liu, Renchu Guan, Fausto Giunchiglia, Yanchun Liang, and Xiaoyue Feng. 2021. Deep attention diffusion graph neural networks for text classification. In *EMNLP*. 8142–8152.
- [31] Yonghao Liu, Lan Huang, Fausto Giunchiglia, Xiaoyue Feng, and Renchu Guan. 2024. Improved Graph Contrastive Learning for Short Text Classification. In *AAAI*.
- [32] Yonghao Liu, Mengyu Li, Wei Pang, Fausto Giunchiglia, Lan Huang, Xiaoyue Feng, and Renchu Guan. 2025. Boosting Short Text Classification with Multi-Source Information Exploration and Dual-Level Contrastive Learning. In *AAAI*.
- [33] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).
- [34] Aditya Krishna Menon, Sadeep Jayasumana, Ankit Singh Rawat, Himanshu Jain, Andreas Veit, and Sanjiv Kumar. 2020. Long-tail learning via logit adjustment. In *ICLR*.
- [35] Fausto Milletari, Nassir Navab, and Seyed-Ahmad Ahmadi. 2016. V-net: Fully convolutional neural networks for volumetric medical image segmentation. In *3DV*. 565–571.
- [36] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. In *NeurIPS*.
- [37] Joshua David Robinson, Ching-Yao Chuang, Suvrit Sra, and Stefanie Jegelka. 2020. Contrastive Learning with Hard Negative Samples. In *ICLR*.
- [38] Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, et al. 2022. Bloom: A 176b-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100* (2022).
- [39] Xiaohui Song, Longtao Huang, Hui Xue, and Songlin Hu. 2022. Supervised Prototypical Contrastive Learning for Emotion Recognition in Conversation. In *EMNLP*.
- [40] Sainbayar Sukhbaatar, Joan Bruna, Manohar Paluri, Lubomir Bourdev, and Rob Fergus. 2015. Training convolutional networks with noisy labels. In *ICLR*.
- [41] Varsha Suresh and Desmond Ong. 2021. Not All Negatives are Equal: Label-Aware Contrastive Loss for Fine-grained Text Classification. In *EMNLP*. 4381–4394.
- [42] Jian Tang, Meng Qu, and Qiaozhu Mei. 2015. Pte: Predictive text embedding through large-scale heterogeneous text networks. In *SIGKDD*. 1165–1174.
- [43] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shrutu Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [44] Laurens Van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *JMLR* 9, 11 (2008).
- [45] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *NeurIPS*.
- [46] Peng Wang, Kai Han, Xiu-Shen Wei, Lei Zhang, and Lei Wang. 2021. Contrastive learning based hybrid networks for long-tailed image classification. In *CVPR*. 943–952.
- [47] Jason Wei and Kai Zou. 2019. EDA: Easy Data Augmentation Techniques for Boosting Performance on Text Classification Tasks. In *EMNLP*. 6382–6388.
- [48] Ziang Xie, Sida I Wang, Jiwei Li, Daniel Lévy, Aiming Nie, Dan Jurafsky, and Andrew Y Ng. 2017. Data noising as smoothing in neural network language models. In *ICLR*.
- [49] Jiaming Xu, Bo Xu, Peng Wang, Suncong Zheng, Guanhua Tian, and Jun Zhao. 2017. Self-taught convolutional neural networks for short text clustering. *Neural Networks* 88 (2017), 22–31.
- [50] Yuzhe Yang, Hao Wang, and Dina Katabi. 2022. On Multi-Domain Long-Tailed Recognition, Generalization and Beyond. In *ECCV*.
- [51] Liang Yao, Chengsheng Mao, and Yuan Luo. 2019. Graph convolutional networks for text classification. In *AAAI*. 7370–7377.
- [52] Hongyi Zhang, Moustapha Cisse, Yann N Dauphin, and David Lopez-Paz. 2018. mixup: Beyond Empirical Risk Minimization. In *ICLR*.
- [53] Yifan Zhang, Bingyi Kang, Bryan Hooi, Shuicheng Yan, and Jiashi Feng. 2021. Deep long-tailed learning: A survey. *arXiv preprint arXiv:2110.04596* (2021).
- [54] Xingyu Zhao, Yuxuan An, Ning Xu, Jing Wang, and Xin Geng. 2023. Imbalanced label distribution learning. In *AAAI*.
- [55] Jianggang Zhu, Zheng Wang, Jingjing Chen, Yi-Ping Phoebe Chen, and Yu-Gang Jiang. 2022. Balanced contrastive learning for long-tailed visual recognition. In *CVPR*. 6908–6917.

## A Appendix

### A.1 The Impacts of Different Data Augmentations

To investigate the impacts of different data augmentations on the model, we implement the following three methods. (I) *Word substitution*: We use synonyms from WordNet to replace associated

words in the input text. (II) *Back translation*: We first translate the input text into another language (French), and then translate it back into English to generate a paraphrased version of the input text. (III) *Contextual embedding*: We use pre-trained LMs to find the top- $n$  most suitable words in the input text for insertion. Based on the results shown in Table A.1, we can observe that all data augmentation methods can achieve desirable results. However, there is no universal method that can perform optimally on all datasets, as the optimal configuration of data augmentation methods varies depending on the dataset.

## A.2 Detailed Descriptions of Datasets

**R52** [30] is a dataset containing news articles of 52 categories from Reuters for news classification. **Ohsumed** [26] contains many important medical studies and a bibliographic classification dataset. Notably, this dataset contains only the titles. **TREC** [23] is a question classification dataset that includes six categories of questions. **DBLP** [42] consists of six diverse kinds of paper titles extracted from computer science bibliographies. **Biomedical** [49] is a collection of biomedical paper titles that is used for topic classification. **CR** [11] is a customer review dataset for sentiment analysis, where the reviews are labeled positive or negative.

## A.3 Detailed Descriptions of Baselines

### Pretrained language models and their imbalanced learning versions

(I) **BERT** [10] is trained with a large-scale corpus and encodes implicit semantic information. (II) **BERT+Dice Loss** [24] uses BERT to encode texts and the Dice loss for optimization. The Dice method dynamically adjusts the weights according to the learning difficulty of the input samples. (III) **BERT+Focal Loss** [25] combines BERT, which learns text features, with Focal loss, which offers small scaling factors for predicted classes with high confidence. (IV) **RoBERTa** [33] is an enhanced version of BERT that utilizes more training data and large batch sizes.

### Contrastive learning models

(I) **SimCSE** [14] uses two dropout operations in an unsupervised manner and sentence labels in a supervised manner to generate positive sample pairs. (II) **SCLCE** [15] combines the SCL objective with cross-entropy in the fine-tuning stage of NLP classification models. (III) **SPCL** [39] attempts to address the class-imbalanced task by combining CL and curriculum learning. (IV) **DualCL** [7] solves TC tasks via a dual CL mechanism, which simultaneously learns the input text features and the classifier parameters.

### Imbalanced contrastive learning models

(I) **Hybrid-SC** [46] is a hybrid network architecture that integrates a supervised contrastive loss for image representation learning and a cross-entropy loss for classifier learning. The training process is designed to gradually shift from feature learning to classifier optimization. (II) **BCL** [55] simultaneously balances the gradient contributions of negative classes and ensures the presence of all classes within each mini-batch. This design satisfies the conditions for forming a regular simplex, which facilitates the optimization of the cross-entropy loss. (III) **TSC** [22] first generates a set of uniformly distributed target vectors on a hypersphere. During training, it encourages features from different classes to align with these distinct targets, thereby enforcing a uniform distribution of all classes in the feature space..

### Large language models

(I) **GPT-3.5** [36] leverages a vast amount of texts for self-supervised learning and incorporates reinforcement learning with human feedback techniques to fine-tune the pretrained model. Note that when we conduct the experiments, the GPT-3.5 API is still available. (II) **Bloom-7.1B** [38] is a decoder-only Transformer language model trained on the ROOTS corpus, and exhibits enhanced performance after being fine-tuned with multi-task prompts. (III) **Llama2-7B** [43] is also a decoder-only large language model that is trained on a new mixup of data from public dataset. It also increases the size of the pretraining corpus by 40%, doubles the model's context length, and employs grouped query attention. (IV) **Llama3-8B** [1] is also an auto-regressive language model, adopting a similar model architecture to Llama2. The main difference lies in its pretraining on a dataset containing over 15T tokens.

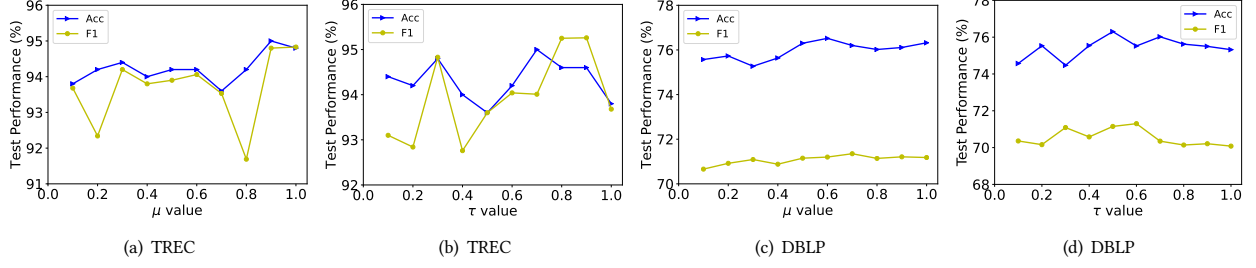
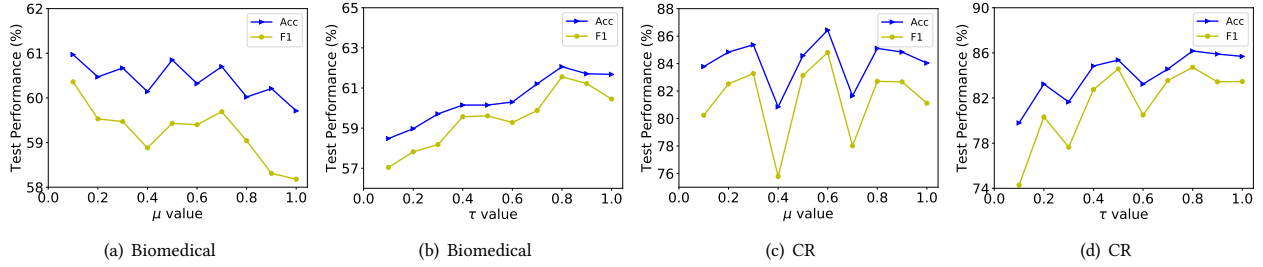
## A.4 Hyperparameter Study

We conduct a series of hyperparameter sensitivity studies with controlled weight  $\mu$  and temperature  $\tau$  on different datasets under  $ir=50$ . From Figs. A.1 and A.2, we observe that the best configuration of the  $\mu$  and  $\tau$  values varies with the different datasets. When the range of  $\mu$  is  $[0.3, 0.7]$ , this often leads to improved performance. For  $\tau$ , taking a value from  $[0.5, 0.9]$  allows the model to achieve satisfactory performance.

We also show the impact of the  $K$  value in the top- $K$  positive samples on the evaluated datasets in the Table A.2, where except for R52 and Ohsumed, the remaining datasets are conducted under the setting of  $ir=50$ . We observe that the model performance first increases and then decreases as  $K$  increases. The model performance can basically reach its peak at  $K=20$ . One reasonable reason is that when the value of  $K$  is too small, the model cannot synthesize diverse hard samples, while when the value of  $K$  is too large, the model may generate simple samples.

**Table A.1: The results of different augmentations on all original datasets.**

| Dataset              | R52   |       | Ohsumed |       | TREC  |       | DBLP  |       | Biomedical |       | CR    |       |
|----------------------|-------|-------|---------|-------|-------|-------|-------|-------|------------|-------|-------|-------|
|                      | Acc   | F1    | Acc     | F1    | Acc   | F1    | Acc   | F1    | Acc        | F1    | Acc   | F1    |
| Word substitution    | 96.11 | 84.83 | 68.44   | 60.82 | 97.20 | 97.63 | 80.15 | 77.59 | 72.52      | 72.65 | 93.09 | 92.51 |
| Back translation     | 95.92 | 84.69 | 68.09   | 59.41 | 97.20 | 96.84 | 80.12 | 77.22 | 72.83      | 72.87 | 93.09 | 92.58 |
| Contextual embedding | 96.05 | 84.76 | 67.62   | 59.20 | 97.80 | 97.18 | 80.14 | 77.38 | 72.61      | 72.61 | 93.09 | 92.54 |

**Figure A.1: Hyperparameter sensitivity of our model on TREC and DBLP under  $ir=50$ .****Figure A.2: Hyperparameter sensitivity of our model on Biomedical and CR under  $ir=50$ .****Table A.2: Results of SharpReCL on the evaluated datasets.**

| Model      | K=10       | K=20              | K=30              | K=40       | K=50       |
|------------|------------|-------------------|-------------------|------------|------------|
| R52        | 95.82±0.22 | <b>96.11±0.36</b> | 96.01±0.25        | 95.08±0.29 | 95.32±0.10 |
| Ohsumed    | 68.02±0.19 | <b>68.44±0.22</b> | 68.19±0.20        | 67.68±0.35 | 67.32±0.16 |
| TREC       | 94.02±0.49 | <b>94.80±0.55</b> | 94.29±0.26        | 93.68±0.32 | 93.36±0.12 |
| DBLP       | 75.22±0.29 | 76.30±0.25        | <b>76.59±0.22</b> | 76.08±0.39 | 75.92±0.26 |
| Biomedical | 61.02±0.19 | <b>61.56±0.28</b> | 61.09±0.20        | 60.68±0.45 | 60.52±0.16 |
| CR         | 83.62±0.15 | <b>84.04±0.25</b> | 83.74±0.19        | 83.02±0.20 | 82.64±0.22 |